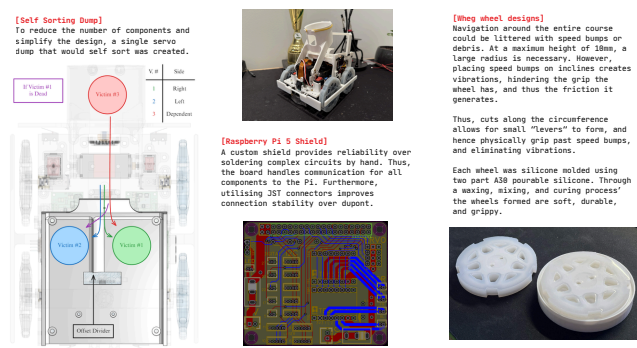
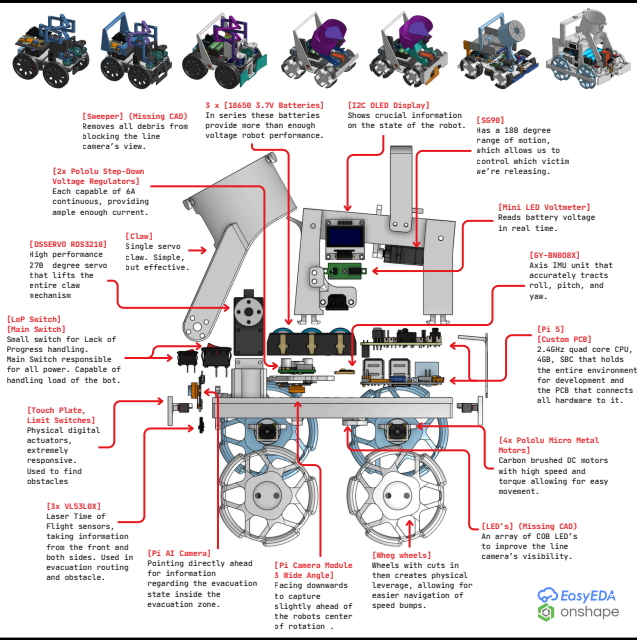




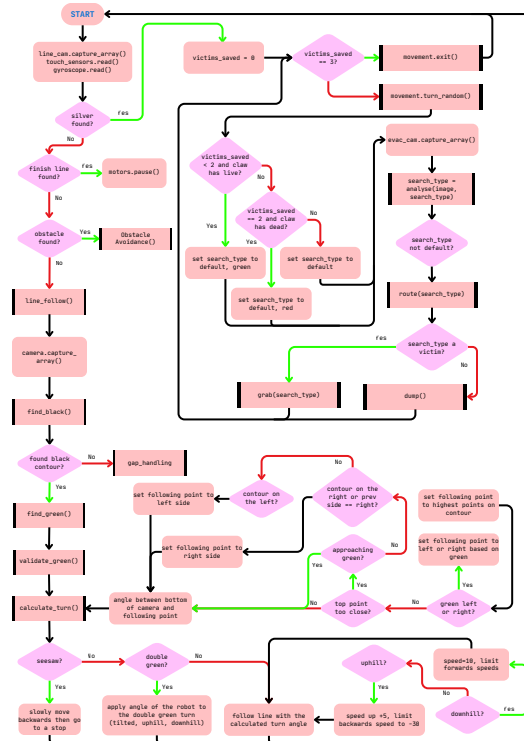
[TEAM]

League: Robocup Junior Rescue Line
 Region: New Zealand

[Frederick] Team Captain
Lead CAD designer
3D printing and assembly
Kiax work & 5th/6th year
Version control, document creation and submission
Programming:
• Victim identification
• Program control loop
• Sensor modules

[Amaan] PCB Designer
Custom shields and sensors boards
Lead programmer:
• Line follow
• Intersection
• Obstacle avoidance
• Line detection
• Evasion zone detection
• Exit detection
• Decline/Tilt detection

[Results]
 (National) 2018 Auckland Secondary
 (National) 2018 New Zealand Premier
 (National) 2019 Australia Open Tertiary
 (National) 2021 New Zealand Self Driving Car Senior
 (Regional) 2022 Auckland Premier
 (National) 2023 New Zealand Premier
 (National) 2023 Australia Open Tertiary
 (National) 2024 Singapore U19 International
 (National) 2024 Australia U19 International
 (National) 2025 Japan U19 International
 (National) 2025 Singapore U19 International
 (International) 2025 Brazil U19 International
 (National) 2025 Australia U19 International



[Live Victim Identification]
 We take advantage of the consistency of the playing field: there are always spectral highlights due to bright overhead lights, that always appear on the live victims. Thus, filtering for bright values from the camera quickly identifies live victims. This helps with removing false detections from the custom AI model.

Live victim pseudocode
 1.capture image
 2.run custom ai model
 3.filter for bright pixels
 4.find contours
 5.verify size, position
 6.return x

[Dead Victim Identification]
 In the past, filtering for the black victim was inconsistent at best: the victim from outside the course, especially considering the wilderness of the evasion zone, created many dark spots as the robot faced the outside. A custom AI model alongside legacy methods helped with uncertainty.

[Tilted boards and incline]
 Our older robots used mercury-tilt switches that were nothing more than on-off contacts. They chattered on every bump, needed separate sensors for non-up and non-down. By replacing them with a single MPU6088 IMU we now read yaw, pitch, and roll angles, giving us more information.

[Continuous pitch data]
 With continuous pitch data we set a simple threshold—about 45 degrees—to decide when we are climbing or descending. The moment pitch rises past that limit the code increases base speed to clear speed bumps; when pitch drops below -11 degrees it softens the steering gains and slows the drive so the robot doesn't slide downhill. Crossing from positive to negative pitch within a second flags a sensor, prompting a short pause until the board settles before moving on.

pitch = imu.get_pitch();
 if pitch > +11: speed = 30 # uphill
 elif pitch < -11: speed = 15 # downhill
 else: speed = 25 # flat
 if sign_flip(pitch, 1):
 stop_motors();
 sleep(1)

note: There is more complexity behind sideways tilted ramps which are apparent in code but not fully represented in the above pseudocode.